DIKU NLP Course 2026: Group Project
Reliable Multilingual Question Answering

Last updated September 1, 2026
This project description may be slightly adapted throughout the course. We will notify everyone on Absalon in case of such changes.

The aim of the project is to investigate one central question:

> **How reliably can a multilingual question answering system extract an answer from an English context and abstain when that context does not support an answer?**

You will develop an incremental system for questions in Arabic, Korean and Telugu, compare several NLP approaches, and analyse when they succeed or fail. The project follows the course syllabus; each stage below is introduced alongside the corresponding course material.

Organization. The project will be graded as a whole and all core stages are mandatory. Complete the project in groups of up to **3 students**.[1] The core pipeline is shared. In addition, each member must take primary responsibility for one distinct extension in Part 7. Your report must indicate:

- which core components were completed jointly or individually, and who took primary responsibility for each extension;
- motivated explanations for your decisions;
- a human-readable description of what you implemented;
- your results, error analysis and limitations;
- whether and how you used generative AI, using the standard template;[2] and
- if you used generative AI, complete logs of all project-related interactions with each system (prompts and outputs). Include them as PDF or plain-text files in a `genai-logs` directory in the submitted ZIP. You may redact personal data, but mark every redaction. If you did not use generative AI, state this in the declaration; no logs are then required.

Agree on the division of work early. If a group member becomes non-responsive or the agreed division breaks down, contact the course organizer on Absalon as soon as the problem arises and before submission. Keep a brief

[1] You may complete the project alone, but we recommend working in groups.

[2] https://kunet.ku.dk/work-areas/teaching/digital-learning/chatgpt-and-ai/new-rules-and-principles-from-september-2025/template-for-declaration/Pages/default.aspx

dated record of the agreed division, attempts to resolve the problem and work completed by each member. The contribution statement and any contemporaneous documentation will be used to determine individual contributions; do not wait until after the deadline to report a collaboration problem.

Format. Submit one non-anonymized PDF using the ACL template[3] and its `final` option. The page limit is $3 + k$ **pages**, where k is the number of group members: 4 pages for an individual report, 5 pages for a pair and 6 pages for a group of three. The limit includes figures and tables; content after the applicable limit will not be graded. References, a reproducibility appendix, the contribution statement and AI declaration do not count towards the limit. The appendix may contain hyperparameters, seeds and implementation details, but not new results or arguments needed to support the report's main claims. Do not include a separate title page. An abstract is not required or expected. Write the report in English. Submit your code, challenge set and any required generative-AI logs in one ZIP file.

Feedback checkpoints. Ungraded feedback is available through Absalon: every Monday in weeks 36-41, you have the option to submit your full report (of course, with only the parts you have done until the point of submission). Submit just one report per group. You will get feedback on it so you can improve it for the final submission.

Final submission. Submit the report and ZIP file before **30 October 17:00 Copenhagen time** on Digital Exam; do not submit the final version on Absalon. Submit one report per group.

Infrastructure. We recommend Google Colab,[4] which provides free CPU and GPU resources. You should not need a paid account. Start with a small sample and verify the complete pipeline before scaling up. Record the hardware, software versions, random seeds, training time and number of training examples for every learned model. If you reduce the data or model size to meet Colab limits, apply a fixed sampling procedure and justify it.

Dataset. TyDi QA (Clark et al., 2020) is a multilingual question answering dataset. We use a combined and simplified version of the Reading Comprehension variant of XOR QA (Asai et al., 2021) and XOR-AttriQA (Muller et al., 2023):

`https://huggingface.co/datasets/coastalcph/tydi_xor_rc`

Questions are in several languages, while contexts and answer spans are in English. Some questions are *unanswerable*: their answer is not supported by the supplied context. Use only Arabic (`ar`), Korean (`ko`) and Telugu (`te`) for the mandatory training and validation experiments. The sparse `answer_inlang` field is not part of the core task; it may be used for a suitably qualified extension.

[3] `https://github.com/acl-org/acl-style-files`
[4] `https://colab.research.google.com/`

1 Shared Evaluation Protocol

Use the supplied training split to fit models and the validation split for model selection, including thresholds. Report results both overall and separately for Arabic, Korean and Telugu, always with the number of evaluated examples. For integrated QA, define a no-answer score, such as the null/CLS-span score minus the best non-empty-span score. The no-answer threshold is the validation-tuned cutoff on this score that determines whether the system returns an empty answer; state which side of the cutoff means no answer.

- For **answerability**, report macro-averaged F1 (the unweighted mean of answerable-class and unanswerable-class F1) and precision/recall/F1 for both classes. Accuracy may be included but is insufficient because the classes are imbalanced.
- For **BIO span labelling**, report token-level answer-token F1, treating B and I tokens as positive and O tokens as negative, and exact span match. State exactly how unanswerable examples enter each metric.
- For **integrated extractive QA**, report SQuAD-v2 exact match and token F1, plus answerability macro-F1 at the chosen no-answer threshold.

Use the same data splits and evaluation code for models being compared. Include at least one deliberately simple, interpretable baseline for every learned task. Report more than one random seed when stochastic variation could change a central conclusion, or explain why this was infeasible. Do not use the challenge set in Part 8 for model selection or prompt development.

2 Week 36: Data and Rule-Based Baselines

Download the dataset and inspect its columns. Report Item 1 separately by language and split, plus overall example counts and answerability proportions. Compute Item 2 from the training questions separately by language. Apply Item 3 to every answerable example in both splits.

1. report the number of examples, answerable/unanswerable proportions, median and interquartile range of tokenised question and context lengths (a table is sufficient; plots are optional), missing values and exact duplicate question–context pairs;
2. report the five most common question tokens and their counts for each language, together with an English translation, and explain your tokenisation; and
3. verify programmatically that every answerable item's `answer` equals the substring beginning at `answer_start`, and report the number checked and any failures.

Implement and evaluate two answerability baselines: (i) the majority-class baseline estimated from the training split and (ii) a deterministic rule-based classifier that uses only the question and context. The rule may use tokenisation, lexical features or machine translation, but no labelled validation examples or trained answerability/QA model. Discuss what information the rule can and cannot exploit in this cross-lingual setting.

3 Week 37: Structured Span Prediction

Convert the character-level answer offsets into BIO labels over context tokens. Add automatic checks for at least the following cases: an answer at character 0, a multi-token answer, punctuation adjacent to an answer and an unanswerable example. Document how subword pieces are handled if applicable.

Implement one **question-conditioned** sequence labeller for the group: the representation of the question must influence the predicted label for every context token. Compare it with a simple span baseline. An empty-output baseline is sufficient. If you use lexical overlap, select context tokens using only overlap with the question (optionally after fixed preprocessing or translation), convert the best contiguous run to a span and never use gold answer text or offsets. The correct output for an unanswerable question is an empty span. Evaluate and analyse the models according to Section 1.

4 Week 38: Language Modelling

Each group member must implement one language-model experiment on questions in one of Arabic, Korean or Telugu; use different languages within a group whenever possible. An experiment may use an n-gram or neural language model and must include an explicit tokenisation and unknown-token strategy. Members may use the same model family, tokenisation and unknown-token strategy when appropriate; these choices do not have to be distinct. Each member must still run and interpret one language-specific experiment. Train only on questions from the training split and report per-token perplexity on the corresponding validation questions.

Inspect examples with high and low perplexity and formulate one hypothesis about what the model captures. Raw perplexities are not directly comparable when models use different token units or vocabularies. Do not make a cross-language perplexity claim unless the normalisation makes the comparison valid; otherwise compare models only within the same language and representation.

5 Week 39: Learned Answerability

Implement two classifiers that receive both question and context:

1. an interpretable learned baseline, such as logistic regression using bag-of-words counts, overlap, length or translation-derived features; and
2. a recurrent neural classifier using learned or pretrained representations.

Complete all of the following:

- motivate the representations and the way question and context information interact;
- evaluate both models under the shared protocol and compare them with the Week 36 baselines;
- manually inspect at least ten validation errors, including false positives and false negatives from all three languages where available, create categories for recurring failure types, and report category counts with representative examples; and

- if you use the Week 38 language-model score as a feature or diagnostic, explain why it should be informative.

6 Weeks 40–41: Integrated Multilingual QA

Fine-tune one multilingual Transformer QA model that receives a non-English question and English context and predicts either (i) start and end positions of an English answer span or (ii) no answer. Train a single joint model on Arabic, Korean and Telugu unless you have a clearly motivated alternative. State how the model scores no answer, then choose the no-answer threshold defined in Section 1 using validation data only.

Evaluate the integrated model according to Section 1. Compare its span predictions with the Week 37 sequence labeller and its abstention decisions with the Week 39 answerability classifiers. Analyse at least ten validation errors, using a linguistically or task-motivated error taxonomy rather than only grouping cases as short/medium/long inputs or low/medium/high model confidence. Include examples from all three languages.

7 Week 43: Individual Extensions

Each member must take primary responsibility for one distinct extension. State a falsifiable hypothesis before running it and compare the extension with the relevant core model under the same evaluation protocol. Suitable extensions include:

- translating questions to English before applying an English QA model;
- calibrated abstention or an alternative no-answer thresholding method;
- continued language-model pretraining before QA fine-tuning;
- zero-shot transfer to a language excluded from core training;
- an alternative question–context interaction or span architecture; or
- in-language answer generation using the sparse `answer_inlang` subset, with its sample size and imbalance treated explicitly as limitations.

You may propose another extension, but it must add a research comparison rather than only computational scale. Data augmentation, retrieval-augmented generation and similar advanced methods are acceptable only after the complete core pipeline and baselines work. A formal significance test is not required: state the expected direction and metric before running the extension, then report the observed difference on the same examples and variation across seeds or paired examples when feasible. Use the term "statistically significant" only if you apply an appropriate test.

8 Weeks 43–44: Paired Challenge Set

Create a `challenge.json` JSON array and include it with your code. Each member must contribute exactly five matched pairs (ten records): one answerable and one minimally altered unanswerable question per pair, using the same English context. The alteration should make abstention difficult without making the question nonsensical. Questions may be in Arabic, Korean, Telugu or

another language; using an unseen language constitutes zero-shot evaluation and must be discussed.

Every record must contain:

`id`, `pair_id`, `created_by`, `question`, `context`, `lang`, `answerable`, `answer_start`, `answer`, and `source_url`.

Identifiers must be unique. For answerable records, `answer` must be an exact context substring and `answer_start` its zero-based character index. For unanswerable records, use an empty answer string and `answer_start` $= -1$. The two records in a pair must have the same context, language, creator and source URL, and exactly one must be answerable. Use a public source and record its URL. Download the student starter and run `validate_challenge.py`; invalid records will not be graded.

Freeze all models and thresholds before running them on the challenge set. Evaluate the Week 36 rule, the best Week 39 answerability classifier, the Week 37 sequence labeller, the integrated QA model and each relevant extension. Report paired results and discuss whether failures arise from the language, the minimal question alteration, answer extraction or abstention.

9 Report Structure and Grading

Organise the report as a coherent investigation rather than one section per teaching week. A typical structure is: research question and data; methods; validation results; challenge-set and error analysis; extensions and limitations; conclusion. The title must state the group name and all members. Put a concrete contribution statement last.

The assessment is based on the report and verified code, using the following weighting:

- **45% Core methods and correctness**: complete, appropriate implementations and baselines from Weeks 36–41;
- **15% Individual extensions**: clear hypotheses, justified designs and fair comparisons;
- **25% Evaluation and analysis**: correct metrics, controlled comparisons, challenge-set quality, error analysis and limitations;
- **10% Reproducibility**: executable code, validation checks, seeds, environments, data handling and resource reporting; and
- **5% Communication**: clear, concise and well-supported presentation.

Implementations earn credit only when described in the report. Raw scores without interpretation are insufficient. Advanced methods cannot compensate for a missing mandatory baseline. Individual grades may differ when the contribution statement or contemporaneous documentation shows materially different contributions.

10 Academic Code of Conduct

You may discuss the project with other students, but sharing code or report text between groups is not permitted. Copying code, text or ideas without

acknowledgement will be treated according to the University's plagiarism regulations. Acknowledge lecture/lab code, external implementations, models, datasets, translation systems and other sources. Ask project questions on the Absalon discussion forum.

Generative AI. Generative AI may be used for brainstorming, language editing, translation, code generation and debugging. You remain responsible for understanding, testing and verifying every submitted line of code, claim and result. It may not replace the required experiments or analysis, fabricate results or citations, or receive confidential or personal data. Declare every use and submit complete logs as required under Organization; undeclared AI-assisted code or text is not permitted.

References

Akari Asai, Jungo Kasai, Jonathan Clark, Kenton Lee, Eunsol Choi, and Hannaneh Hajishirzi. 2021. XOR QA: Cross-lingual open-retrieval question answering. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 547–564, Online. Association for Computational Linguistics.

Jonathan H. Clark, Eunsol Choi, Michael Collins, Dan Garrette, Tom Kwiatkowski, Vitaly Nikolaev, and Jennimaria Palomaki. 2020. TyDi QA: A benchmark for information-seeking question answering in typologically diverse languages. *Transactions of the Association for Computational Linguistics*, 8:454–470.

Benjamin Muller, John Wieting, Jonathan Clark, Tom Kwiatkowski, Sebastian Ruder, Livio Soares, Roee Aharoni, Jonathan Herzig, and Xinyi Wang. 2023. Evaluating and modeling attribution for cross-lingual question answering. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 144–157, Singapore. Association for Computational Linguistics.